

Optimizing Adaptive Blending Units

Joshua Feldman
Skidmore College
Saratoga Springs, NY, 12866
4/12/2026

Abstract:

Adaptive Activation Functions (AAFs) offer a promising alternative to fixed nonlinearities in deep neural networks by allowing networks to learn activation shapes during training. One such method, Adaptive Blending Units (ABUs), constructs activation functions as trainable linear combinations of multiple base activations, enabling layer-specific optimization of nonlinear transformations. Although ABUs have demonstrated improved validation performance across several architectures, their adoption may be limited by increased computational cost due to additional activation evaluations and trainable parameters.

This paper introduces Optimized Adaptive Blending Units (OABU), a computationally efficient variant of ABU designed to reduce training and inference overhead while preserving adaptive behavior. Three optimizations are proposed: (1) removal of the identity activation from the blending set, (2) gradient-based freezing of blending weights once stabilization is detected, and (3) dynamic hard-freezing to a single dominant activation when its learned weight exceeds a predefined threshold. These modifications reduce both trainable parameters and forward-pass computations over time.

A convolutional neural network was trained on the CIFAR-10 dataset to evaluate functionality and performance. Experimental results demonstrate that OABU successfully learns stable activation blends, frequently freezing within early epochs, while maintaining competitive classification accuracy. Benchmark comparisons based on a small sample size show measurable reductions in training time, inference latency, and total trainable parameters relative to standard ABU implementations. These findings suggest that adaptive activation functions can be made computationally practical without sacrificing flexibility, moving trainable nonlinearities closer to viable deployment in large-scale deep learning systems.

Part 1: Activation Functions and Adaptive Blending Units

In the past few decades, much has changed in the way data scientists and machine learning engineers develop, build and deploy deep neural networks. Researchers have developed the use of convolutional layers, pooling layers, dropout, data augmentation, residual blocks, transformers, multi head latent attention, and a veritable army of different improvements to the classical neural network. The activation function is no exception to these improvements.

The activation function lies at the heart of the universal function approximating abilities of neural networks. By passing data through an activation function, we can efficiently map non linear functions. Choosing one activation function over another can have a profound effect on the performance of a neural network. Researchers Valadmir Kunc and Kiri Klema at the Czech Technical University published an itemized list of over 500 of these activation functions, ranging from the well known sigmoid, tanh and ReLU to the rarest of functions. In their own words, “There are many activation functions proposed in the literature in the last three decades — some more computationally complex or with higher performance than others.”¹ Each activation function has its own pros and cons, which can make choosing the appropriate one difficult. Early functions such as sigmoid and tanh (hyperbolic tangent) had issues with saturation, in which a neuron largely outputs close to its asymptotic edge.

¹ V. Kunc and J. Kléma, “Three decades of activations: A comprehensive survey of 400 activation functions for neural networks,” *arXiv*, preprint arXiv:2402.09092, Feb. 2024, doi: 10.48550/arXiv.2402.09092

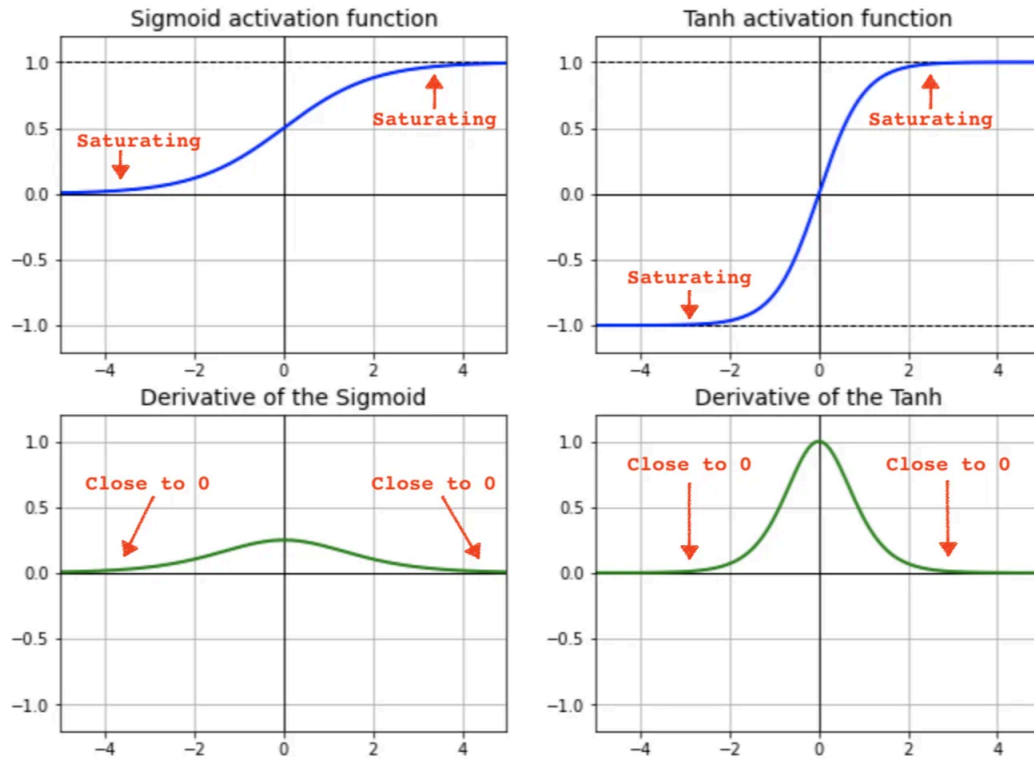


Figure 1: Saturation in Sigmoid and Tanh²

In the diagram above, the flat asymptotic edges in the graphs of sigmoid and tanh, result in very visually flat sections in their derivatives. This can lead to the gradient being close to 0 which also reduces training to close to 0. There are other activation functions that avoid this issue. For example, the most common activation function in deep neural networks today is the Rectified Linear Unit, also known as ReLU. ReLU is formed from the equation $f(x) = \max(0, x)$ and its derivative is 0 when $x < 0$ and 1 when $x > 0$.

²Chen. B, "What do you mean by saturation in the context of neural network training? Discuss the problems associated with saturation," AIML.com, Apr. 2025. [Online]. Available: <https://aiml.com/what-do-you-mean-by-saturation-in-the-context-of-neural-network-training-discuss-the-problems-associated-with-saturation/>

In their paper *Evolutionary Optimization of Deep Learning Activation Functions* researchers at the University of Texas in Austin wrote that, “The choice of activation function can have a large effect on the performance of a neural network. While there have been some attempts to hand-engineer novel activation functions, the Rectified Linear Unit (ReLU) remains the most commonly-used in practice.” What makes ReLU so attractive as a choice for activation function is that it is both computationally simple, and it avoids the saturation problem. However, ReLU has its own issue known as the dying ReLU problem. When x is less than 0, its gradient is also 0, which means that for any negative input, a ReLU based neuron will not be learning.³

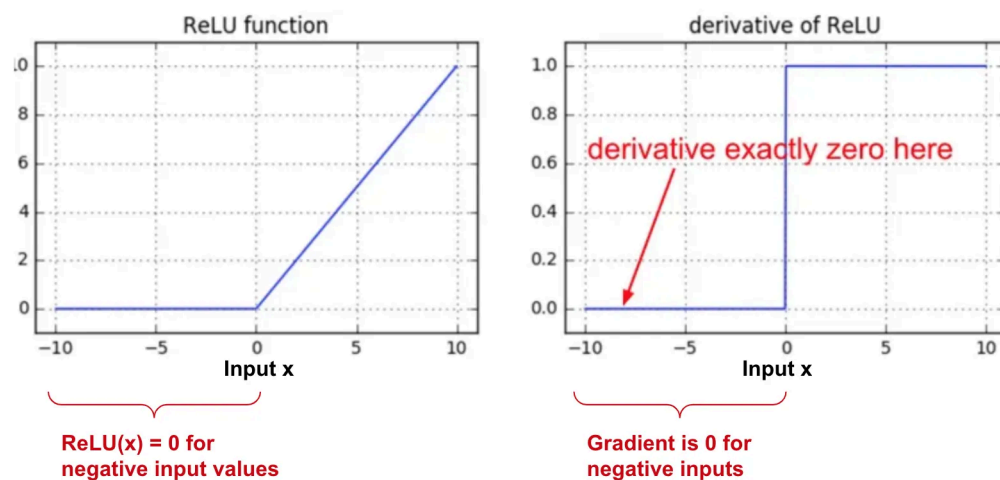


Figure 2: ReLU and its Derivative⁴

³ S. C. Douglas and J. Yu, "Why RELU Units Sometimes Die: Analysis of Single-Unit Error Backpropagation in Neural Networks," 2018 52nd Asilomar Conference on Signals, Systems, and Computers, Pacific Grove, CA, USA, 2018, pp. 864-868, doi: 10.1109/ACSSC.2018.8645556.

⁴ What is the “dead relu” problem and, why is it an issue in neural network training? AIML.com. (2024, July 22). <https://aiml.com/what-is-dying-relu-or-dead-relu-and-why-is-this-a-problem-in-neural-network-training/>

To avoid the dying ReLU problem one method data scientists will often use is different modifications of ReLU. The most common of these is known as leaky ReLU, which uses the equation $f(x) = \max(\alpha x, x)$ where α is some arbitrary value such as 0.01. This adds a slope to the negative side of the function, allowing for training even if all inputs to a leaky ReLU neuron are negative. There are other ReLU variations such as PReLU which uses a trainable α to decide the slope of the left side of the function, and RReLU which uses random values when $x < 0$. Beyond the ReLU family of activation functions there are more activation functions than a data scientist or machine learning engineer could possibly be familiar with. Perhaps the most optimal one could be hidden inside of the list of over 500 functions that Kunc and Klema compiled.

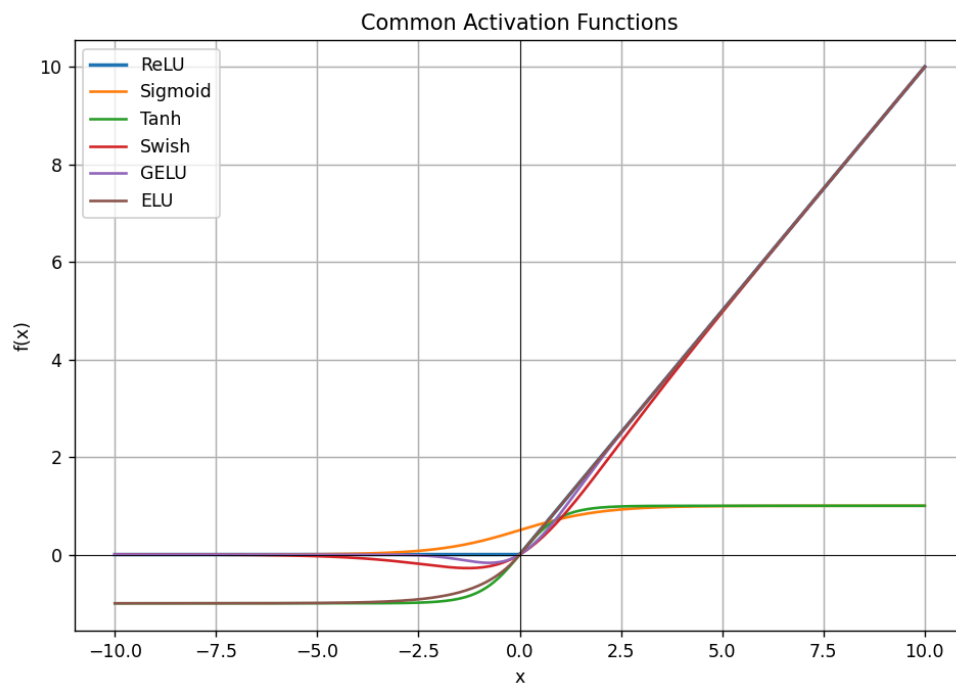


Figure 3: Six Common Activation Functions

Choosing the correct one may never occur, or may take valuable time and energy. For any problem complex enough to require a deep neural network, there is no realistic way for a data scientist to predict with any certainty which activation function is optimal. There are so many activation functions that it is simply unrealistic to even know them all. Oftentimes, data scientists have to play a game of trial and error to find the correct activation function. They may start with ReLU, and run into the dying ReLU problem. They may then switch to leaky ReLU and find the training to be too slow. They may then switch to an activation function known as swish, and find that it works best, or even switch to the lesser known parametric swish⁵. The researchers may have found the best activation function for their particular model on their particular data set, but they lost valuable time and resources picking, and any small change to their model could theoretically cause them to have to start the process of picking an activation function all over again.

Perhaps the optimal activation function for a model could be one that is undiscovered, or unique to the model. Some data scientists believe this is true, which is why they created a new family of activation functions known as Adaptive Activation Functions or AAFs. AAFs train themselves to find the optimal activation function shape. Kunc and Klema outline AAFs in section 4 of their paper.

⁵ M. A. Mercioni and S. Holban, "P-Swish: Activation Function with Learnable Parameters Based on Swish Activation Function in Deep Learning," 2020 International Symposium on Electronics and Telecommunications (ISETC), Timisoara, Romania, 2020, pp. 1-4, doi: 10.1109/ISETC50328.2020.9301059. keywords: {Task analysis;Deep learning;Proposals;Natural language processing;Neurons;Mathematical model;Logistics;activation function;Cifar-10;Cifar-100;Deep Learning;learnable;Machine Learning;Natural Language Processing;ReLU;Swish},

“The activation function introduces non-linearities to neural networks and is crucial for network’s performance. Even though it might be suboptimal, the same activation function is usually used for the whole network or at least for all neurons in a single layer. Over the last few decades, there have been several attempts to use activation functions that might differ across neurons. The adaptive activation functions — i.e., functions that have a trainable parameter that changes their shape — have been receiving more attention recently and might become a new standard in the field.”⁶

One of the listed AAFs in their paper is the “Adaptive Blending Unit” or “ABU” function first introduced by researchers at Osanbruck University in 2018 in the paper, “Adaptive Blending Units: Trainable Activation Functions for Deep Neural Networks”. In the abstract of the paper the researchers outlined their reasons for creating this new function.

“The most widely used activation functions in current deep feed-forward neural networks are rectified linear units (ReLU), and many alternatives have been successfully applied, as well. However, none of the alternatives have managed to consistently outperform the rest and there is no unified theory connecting properties of the task and network with properties of activation functions for most efficient training. A possible solution is to have the network learn its preferred activation functions. In this work, we introduce Adaptive Blending Units (ABUs), a trainable linear combination of a set of activation functions. Since ABUs learn the shape, as well as the overall scaling of the activation function, we also analyze the effects of adaptive scaling in common activation functions”⁷

Adaptive Bending Units use a weighted summation of a variety of activation functions as one large activation function. Each weight is trainable, allowing a model to find the optimal ‘blend’ of activations. The particular blend the researchers chose includes ReLU, tanh, ELU, Swish and the identity function. This gives a wide variety of possible shapes that the blend can take, theoretically finding the optimal activation function. The function is defined as:

⁶V. Kunc and J. Kléma, “Three decades of activations: A comprehensive survey of 400 activation functions for neural networks,” *arXiv*, preprint arXiv:2402.09092, Feb. 2024, doi: 10.48550/arXiv.2402.09092.

⁷L. R. Sütfield and F. Brieger, “Adaptive blending units: Trainable activation functions for deep neural networks,” arXiv:1806.10064, Jun. 2018. [Online]. Available: <https://arxiv.org/abs/1806.10064>

$$f(z_l) = \sum_{j=0}^n a_{j,l} g_j(z_l),$$

Figure 4: ABU Formula

where $g_j(z_l)$ is an activation function from a list of n specifically chosen activation functions and $a_{j,l}$ is a weighting parameter that is trained for each layer l and activation function $g_j(z_l)$.⁸

Thus, each layer of their convolutional neural network had a different blend, as each neuron having its own blend would be far too computationally expensive. Conversely, having just one blend for every neuron in every layer may not be optimal as each layer theoretically maps one set of features of the input data, and therefore may require a different activation function than a different layer mapping a different feature.⁹ ABU activation can also be represented by the formula:

$$\text{ABU}(x) = w_1 * \tanh(x) + w_2 * \text{ReLU}(x) + w_3 * \text{ELU}(x) + w_4 * \text{Swish}(x) + w_5(x)$$

⁸ V. Kunc and J. Kléma, "Three Decades of Activations: A Comprehensive Survey of 400 Activation Functions for Neural Networks," *arXiv*, preprint arXiv:2402.09092, Feb. 14, 2024. doi:10.48550/arXiv.2402.09092.

⁹ M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," arXiv:1311.2901, Nov. 2013. [Online]. Available: <https://arxiv.org/abs/1311.2901>

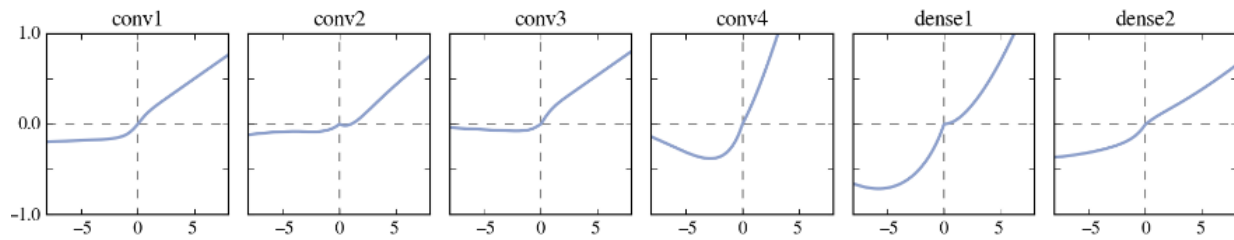


Figure 5: Final ABU Activation Function For Each Layer¹⁰

In figure 5 shown above, each layer found the most optimal blend for their ABU. Some of the blends look like ReLU, Swish, or GELU activations. Some such as the fourth convolutional layer and the first dense layer do not resemble any known activation functions. It would be impossible to select this optimal shape without the help of an AAF such as ABU.

The researchers who published the original ABU paper trained several models with ABU on the CIFAR-10 and CIFAR-100 datasets and found, “robust performance advantages of both approaches over established activation functions in a range of tasks and network architectures.”¹¹ In four out of the six model architectures they trained on CIFAR data the variation with ABU activation had the highest validation. Clearly there is much potential in the use of ABU activation. However, as is often the case with new algorithms and practices there is room for improvement.

¹⁰ L. R. Sütfield and F. Brieger, “Adaptive blending units: Trainable activation functions for deep neural networks,” arXiv:1806.10064, Jun. 2018. [Online]. Available: <https://arxiv.org/abs/1806.10064>

¹¹ IBID

Part 2: Introducing Optimized Adaptive Blending Units

It is clear that there are certain benefits to using ABU activation within a deep neural network. The only issue with ABU activation is the extra computation time required for both training the ABU and the network itself. For each layer in an ABU network there are an extra five computations for every trainable weighted activation function within the blend. Additionally each forward propagation and back propagation there are extra computations for passing inputs into each of the five activation functions in ABU, and their derivatives. It is possible that because of these extra computations that ABU or any other adaptable activation function has by no means become industry standard in deep learning. If a data scientist wanted to use a different ABU blend for every neuron, the performance would be even worse. Neural networks can be very large and time consuming to train, so any addition of time and space complexity would seem unwelcome to data scientists.

It is possible to remove some of the computations from the ABU algorithm. For example, using five blended activation functions may be excessive. Included in the ABU blend is ReLU, ELU, tanh, Swish, and identity. If one activation function could be removed, a likely candidate is the identity function. The identity function is not an activation function normally used in neural networks, as it is simply the equation $y=x$. The researchers who created ABU wrote that, “Deep neural networks are structured around layers, each of which performs a linear transformation of its input before feeding the signal through a scalar non-linear activation function. Chaining larger numbers of non-linear functions then allows the networks to find and extract complex

features in the input. Activation functions thus have a key function in deep neural networks: Without intermittent non-linearities, these networks could only perform linear operations on the input.”¹² As a result, the purely linear nature of the identity function may add little in the way of mapping the non linear functions neural networks need to approximate. While the identity function can act as a learned skip connection, or a scaling variable in very deep architectures, the relatively shallow CNN used in this study already includes batch normalization and pooling, reducing the need for such linear passthrough or scaling behavior. Therefore I thought it would be a good candidate to be removed from ABU altogether.

The lack of utility that the identity function brings to ABU can be seen in the training data of the ABUs the researchers trained (see figure number 6). In every layer the weights of the identity function can be seen dropping precipitously, with little in the way of change for the rest of training. This is because the identity function is a poor activation function, and adds little to the ABUs ability to map complex functions.

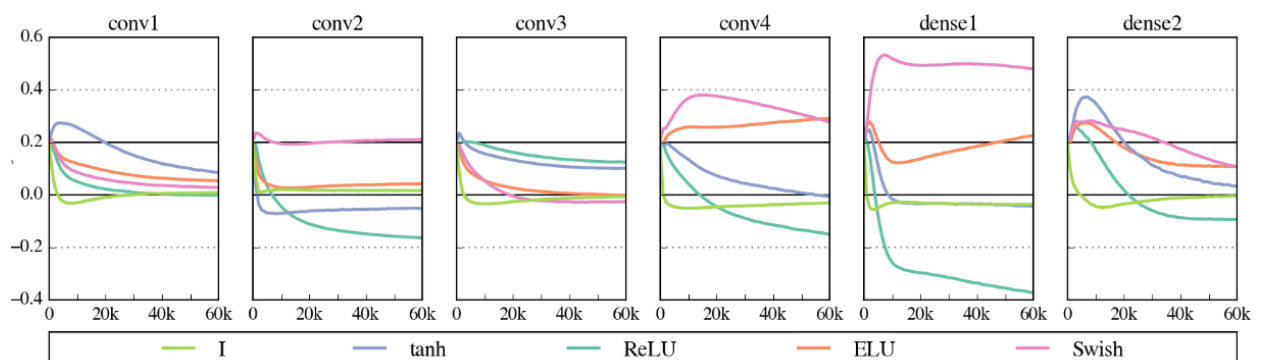


Figure 6: Weights of Each Activation Function in the ABU Over Time During Training¹³

¹² L. R. Sütfield and F. Brieger, “Adaptive blending units: Trainable activation functions for deep neural networks,” arXiv:1806.10064, Jun. 2018. [Online]. Available: <https://arxiv.org/abs/1806.10064>

¹³ IBID

There are further improvements that can be made to ABU activation. As can be seen during the training of some of the layers of the model pictured in figure 6, the rate of change of the activation functions in the ABU reaches a low level, and appears not to be changing much. This is very evident in convolutional layers 1, 2, and 3 as well as dense layer 1. If the training of each weight in the ABU plateaus, why bother to continue to train them? If training plateaus it may be indicative of the optimal activation shape being found. The weights could simply stop training the same way the weights of the neural network itself should stop training if their gradients reach near zero levels. By zeroing gradients of all the weights of the activation functions within the ABU once they have a sufficiently low gradient, the number of computations could be significantly decreased in large scale networks. The only difficulty is setting a threshold for how low the gradient should be, and for how many batches the gradient should have to be below the threshold to stop training the ABU. This is what the experiment will refer to as a “soft freeze” of the ABU weights.

There is one further improvement that can be made to ABU. Let us represent the equation of the optimized ABU by the equation:

$$\text{OABU}(x) = w_1 * \tanh(x) + w_2 * \text{ReLU}(x) + w_3 * \text{ELU}(x) + w_4 * \text{Swish}(x)$$

If w_2 or another weight had a value of .98 after a certain number of epochs, the other three activation functions in the equation would have a weight of less than 0.03. This would mean that ReLU is clearly the dominant activation function, and is therefore the most optimal of the activation functions for the layer this ABU is contained in.

To save computation time and storage time, the model could remove the other three activation functions from the ABU. This is what the experimenter will refer to as a “hard freeze”. After a hard freeze with ReLU as the dominant function, the ABU would then become:

$$\mathbf{OABU}(\mathbf{x}) = \text{ReLU}(\mathbf{x})$$

When the three optimizations above are combined and added into the pre-existing ABU used by its creators, the computation time and storage required to train a deep neural network with ABU activation should be decreased. One concern is that perhaps the accuracy of mapping the optimal activation function would decrease, thereby decreasing the validation accuracy of the overall neural network. I report on the experiments we conducted to determine whether or not the changes are truly improvements.

Part 3: Experimentation

In order to test the effectiveness of the three optimizations to ABU activation, a similar experiment to the one original ABU paper was designed. A CNN with four convolutional layers

and one dense layer was designed, and coded in Python with the PyTorch library. The CNN also included dropout of 0.5, batch normalization, max pooling and the Adam optimizer. The CNN was then trained on CIFAR-10 data, to be later compared for accuracy and computational time against CNNs with the same architecture but different activation functions. Originally the CNN was built using Tensorflow as it is often the industry standard, but the second optimization of freezing the gradient of just the ABU weights proved too difficult to implement.

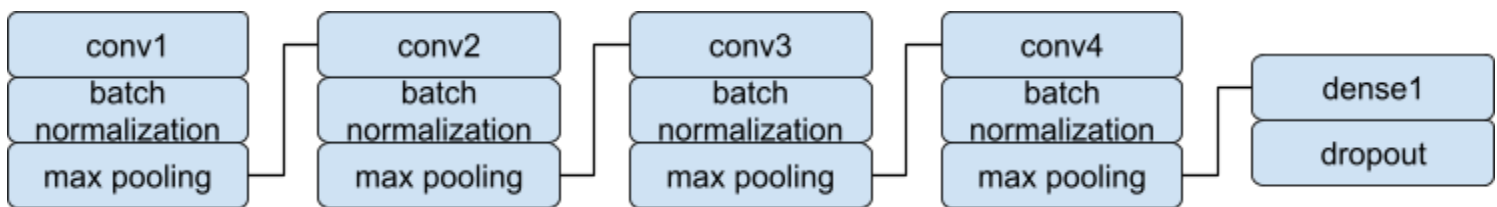


Figure 7: Architecture of the Experimental CNN

The dimensions for the CNN for each experiment included 64 nodes in the first convolutional layer then 128, 256, and 512, followed by a ten node dense layer. Each

convolutional layer has three 3x3 kernels, and padding of 1. This results in 1,554,554 trainable parameters, not including the extra 20 parameters for the OABU, or 25 for ABU.

For the optimized ABU, the threshold for freezing the weights was 0.001 and the number of batches that the gradients of the weights had to be under this threshold to freeze was 30. This threshold was also used to determine if a 'hard freeze' occurs, if one activation function is clearly dominant. The threshold for a hard freeze was set to .85, so that if any weight in the blend reached over .85 the other activation functions and their associated weights would be eliminated from that particular ABU

After some trial and error, with much debugging, the optimized CNN functioned well. The optimized ABU CNN was tested 15 times. Typically, after just one epoch the CNN had frozen the gradients of all of the ABUs, and reached about 45-55% accuracy on CIFAR-10 classification data. Only one 'hard freeze' was observed under the original hyperparameters, with ReLU being the dominant activation function. These 15 successful tests proved that the optimized ABU functioned not only to train the overall CNN, but that the optimizations functioned as intended with all intended freezes of ABU weights occurring.

Part 4: Results and Analysis

The speed at which the optimized ABU CNN trained and inferred was observed to be consistently faster than the standard ABU in both training batches and inferences after training. The CNN with optimized ABU activation on average was 10.651% faster during each training batch and was on average 2.369% faster while inferring. This indicates a significant increase in performance for the optimized ABU.

Average Training Time per Batch (milliseconds)

Activation	Test 1	Test 2	Test 3	Test 4	Test 5	Mean	Standard Deviation
Standard	231.563	223.698	222.339	234.323	220.766	226.537	4.913
Optimized	204.589	198.592	200.818	213.799	194.247	202.409	6.030

Test Time per Batch (milliseconds)

Activation	Test 1	Test 2	Test 3	Test 4	Test 5	Mean	Standard Deviation
Standard	96.715	100.602	97.688	101.199	96.850	98.610	1.742
Optimized	93.137	96.607	97.813	100.731	93.085	96.274	2.656

The reason for the better performance of the optimized ABU is simple; the fewer trainable parameters there are in a model, the faster training and inference is. As can be seen in

figure 10, the optimized ABU starts off slightly faster than the standard ABU. This is due to the fact that for every layer, the optimized ABU has only four ABU weights to train and compute while the standard ABU has five. Then after about 100 batches there is a spike in training time, as the model has decided to freeze the weights of the first layer's ABU. This is subsequently followed by a large drop in computation time per batch, with more spikes indicating additional freezes of ABUs in other layers. In this example test there were no 'hard freezes' of the ABU, which would have sped up the optimized ABU even further.

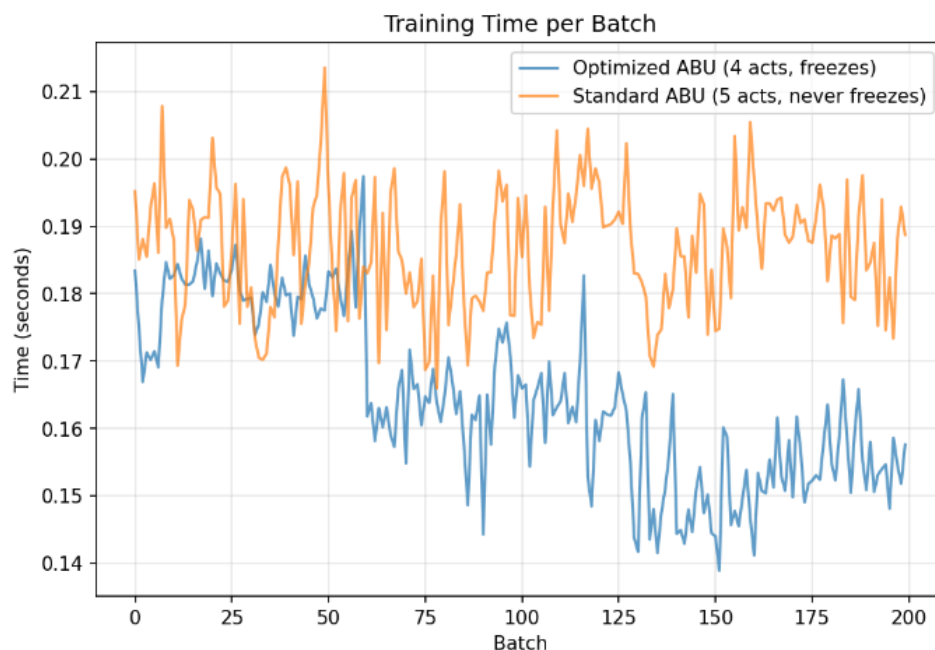


Figure 10: Comparing Training Time per Batch to the Number of Trainable Parameter

There was a higher standard deviation in the results for OABU than ABU. This is indicative of the more unpredictable nature of OABU. With OABU a layer during training may have a soft or hard freeze on a batch or keep learning, which affects the training. Additionally a layer with a hard freeze has fewer computations to make per layer as it only has one weighted activation function in the activation blend. The uncertainty and unpredictability of these freezes results in a higher standard deviation. Perhaps this behavior should be studied more.

As stated above and in figures 9 and 11, the inference time of the optimized ABU was almost always faster than that of the standard ABU. Training may take up much of the resources of data scientists and machine learning engineers, but in order for a deep neural network to be realistically used in the real world, the computation time of a forward phase must be minimized. Many systems such as self-driving cars rely on deep neural networks and convolutional networks to be fast and efficient, so any gain in performance, as little as 2.369% is helpful.

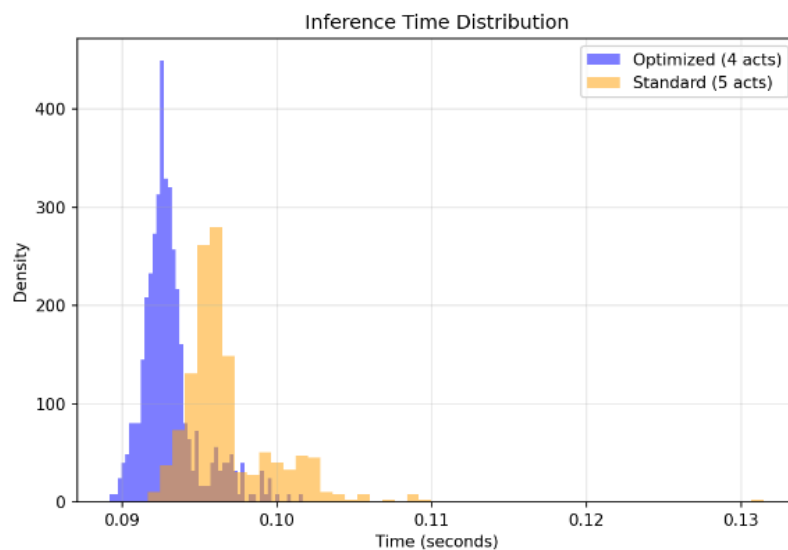
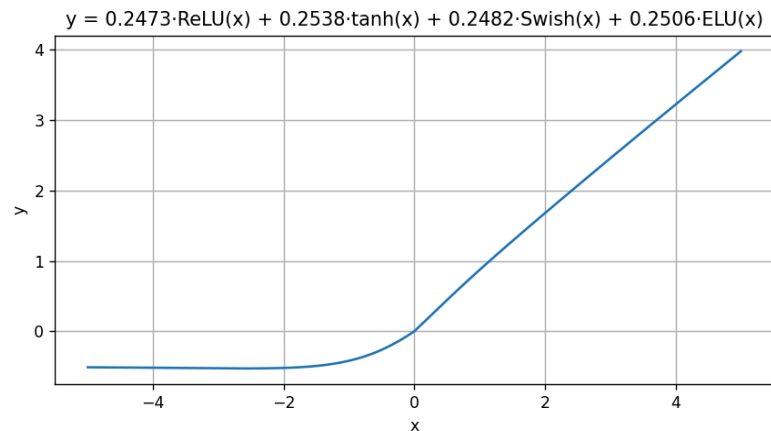
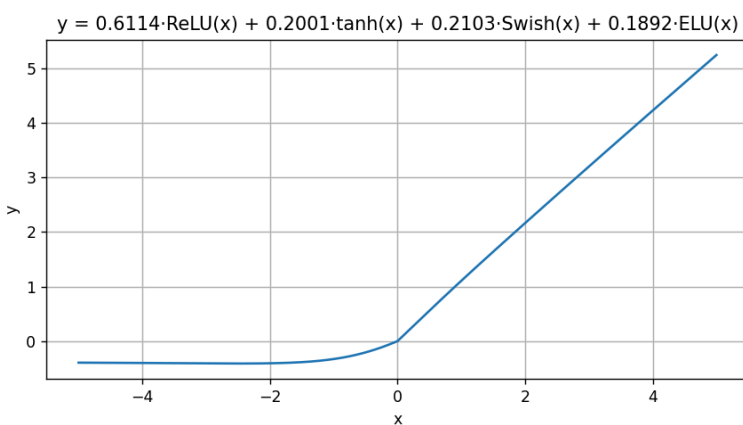


Figure 11: Comparing the Inference Time Between the Optimized and Standard ABU

There was a wide variety of different blends that each ABU layer in the test created. For the most part the weights of each activation function in the blend did not deviate far from a range of 0.10 to 0.40. Occasionally however one weight would dominate the other three and result in a hard freeze.



Figures 12-13: Example Blends from Optimized ABU Tests

A major concern while creating and testing this new activation function would be that its effect on validation accuracy might be negative, and outweigh its effect on computation time during training and inference. The original standard ABU was found to have an advantage over standard activation functions in most cases, but perhaps a change in the ABU could nullify this. The optimized ABU model was compared to two other models in a test, one with a standard

ABU and one with ReLU activation. The three models were first trained on four and then fifty epochs and compared over five tests. None of the models significantly outperformed the others, but the optimized ABU on average had both higher training and validation accuracy than the standard ABU and ReLU models respectively.

Figure 14: Training Accuracy by Activation Function After 4 Epochs

	Test 1	Test 2	Test 3	Test 4	Test 5	Mean	Standard Deviation
Optimized ABU	75.88%	71.16%	71.20%	70.44%	70.33%	71.80%	1.88
Standard ABU	71.69%	71.66%	71.31%	71.51%	71.84%	71.60%	0.16
ReLU	71.20%	70.99%	70.68%	71.39%	71.32%	71.11%	0.23

Figure 15: Validation Accuracy by Activation Function After 4 Epochs

	Test 1	Test 2	Test 3	Test 4	Test 5	Mean	Standard Deviation
Optimized ABU	74.23%	74.56%	75.49%	75.39%	75.15%	74.96%	0.44
Standard ABU	73.57%	74.35%	75.34%	73.63%	74.10%	74.19%	0.58
ReLU	72.69%	74.06%	73.08%	71.62%	75.87%	73.46%	1.30

Figure 16: Training Accuracy by Activation Function After 50 Epochs

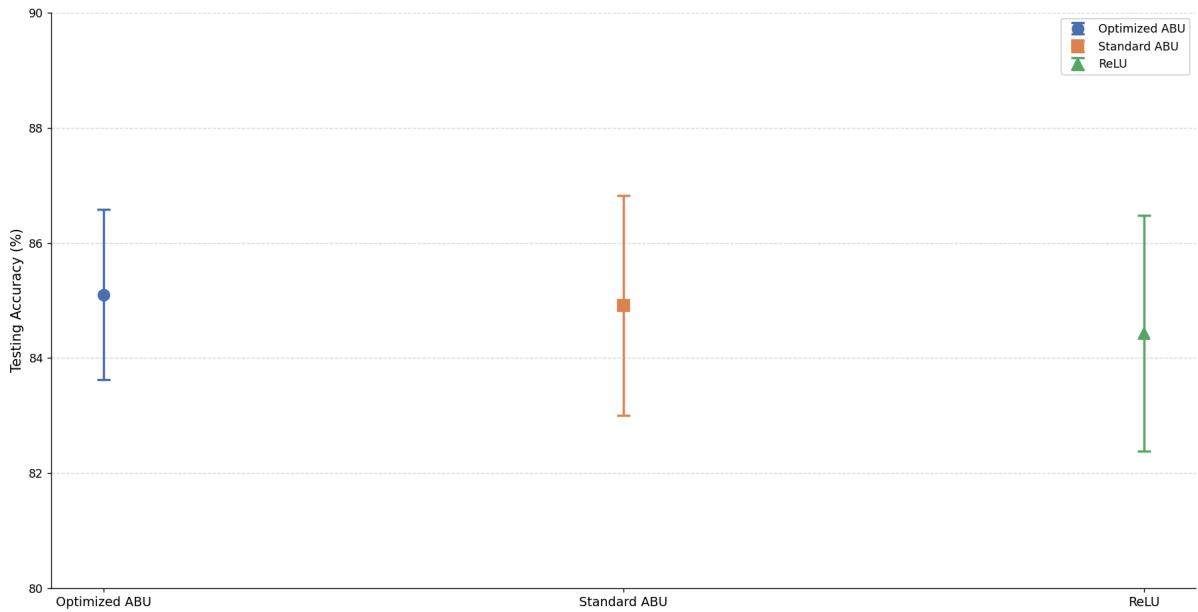
	Test 1	Test 2	Test 3	Test 4	Test 5	Mean	Standard Deviation
Optimized ABU	97.92%	97.27%	97.82%	97.66%	98.02%	97.73%	0.23
Standard ABU	97.68%	97.97%	97.73%	97.21%	97.43%	97.40%	0.25
ReLU	96.20%	96.62%	96.33%	97.03%	96.85%	96.60%	0.28

Figure 17: Validation Accuracy by Activation Function After 50 Epochs

	Test 1	Test 2	Test 3	Test 4	Test 5	Mean	Standard Deviation
Optimized ABU	89.27%	90.08%	89.83%	90.11%	89.91%	89.84%	0.27
Standard ABU	89.86%	89.94%	89.75%	89.64%	89.47%	89.73%	0.15
ReLU	88.53%	89.27%	88.21%	89.19%	88.87%	88.81%	0.36

Figures 18-19: Testing Accuracy by Activation Function After 50 Epochs

	Test 1	Test 2	Test 3	Test 4	Test 5	Mean	Standard Deviation
Optimized ABU	84.20%	85.37%	82.95%	87.41%	85.60%	85.10%	1.48
Standard ABU	83.98%	84.80%	82.18%	88.01%	85.58%	84.91%	1.91
ReLU	84.78%	83.12%	81.21%	86.84%	86.21%	84.43	2.05



While all differences of accuracy were within the margin of error (as there were only five tests) OABU was on average the fastest. This is most likely due to the fact that out of the three activation functions, it was most likely reaching the optimal activation function shape. Also, there

was not a single instance of any ABU blend in any layer of any test in which the identity function had a weight of over 0.03, almost always being the smallest weight in the blend. This further proves that the identity function adds little in the way of mapping the optimal activation function for any layer, and should be removed from the blend altogether.

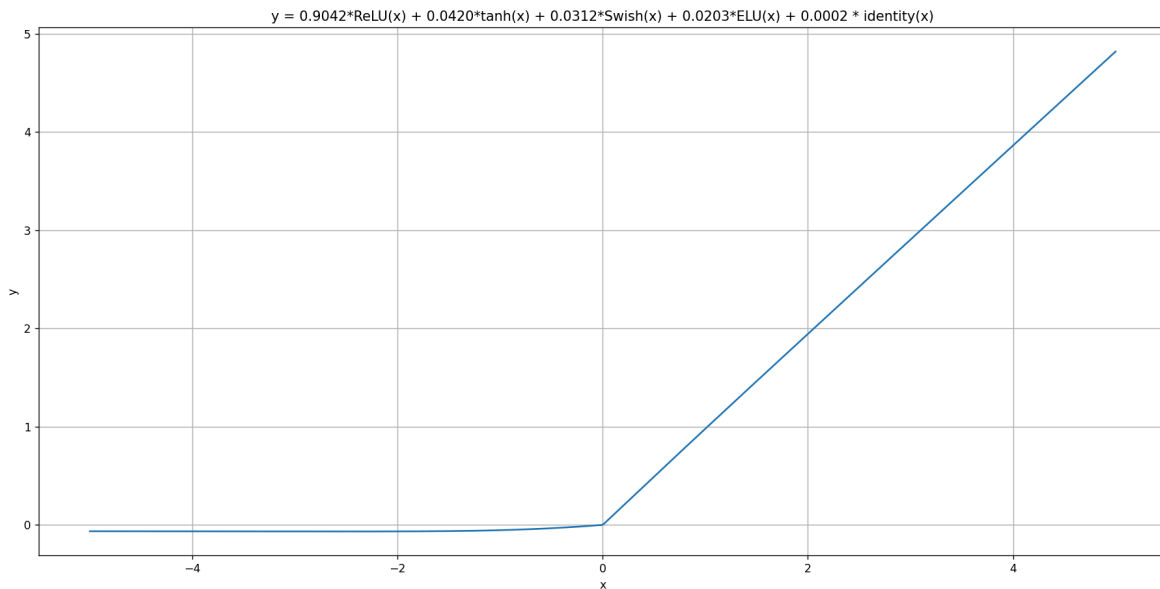


Figure 20: A Standard ABU Blend Indicating a Low Weight for the Identity Function, and Dominant Weight in the Blend

When the threshold for freezing the ABU was decreased to a weight change of less than 0.005 and the iterations of this threshold being consecutively reached to 40, more hard freezes were observed. Once again the dominant functions in the blend during hard freezes were typically ReLU and Swish. The necessity of tuning of this hyperparameter indicated that although the ABU is meant to decrease the amount of time and effort dedicated to tuning hyperparameters such as activation functions, if a data scientist chooses to use optimized ABU

activation in their neural network, they must consider and possibly tune the hyperparameters that freeze the ABU weight.

A higher threshold will result in faster forward and backpropagation but may result in a lower validation accuracy as the optimal blend may not be found. A lower threshold may result in higher validation accuracy but may require more computation time as the ABU is finding the optimal blend under the threshold. Perhaps with more training and experience an accepted default parameter for the thresholds can be found, comparable to how the momentum factor in the nesterov momentum optimizer is typically set to 0.9, or how dropout factors are typically set to 0.5. Of course there is the option of not including weight freezes in the ABU at all, but this will increase the time required to train a deep neural network by adding unnecessary operations to what is already a complex model.

Additionally the threshold for a hard freeze must be set. In this experiment, if a weight in the optimized ABU blend was over .85 then it automatically became the only activation function in the blend, having a weight of 1.00 as a 'hard freeze'. The threshold of .85 was arbitrarily set. Perhaps this is another hyperparameter that has to be tuned, or an accepted default value found.

Figure 21: Example Frozen Optimized ABU Weights In Each Layer After ABU

Hyperparameter Tuning

(Weight Change Threshold of 0.0005, Weight Change Iterations Threshold of 40, Hard Freeze threshold of 0.85)

	ReLU Weight	Tanh Weight	ELU Weight	Swish Weight	Epoch Frozen	Hard Freeze
Layer 1	0.2296	0.6726	0.0334	0.0645	11	No
Layer 2	1.0000	none	none	none	15	Yes
Layer 3	none	none	none	1.0000	14	Yes
Layer 4	none	none	none	1.0000	8	Yes
Layer 5	0.3037	0.3827	0.0663	0.2473	16	No

Part 5: Conclusions

Through experimentation, OABU has been proven to be more efficient than standard ABU. The decrease in training parameters from the removal of the identity function, and the combination of both hard and soft ABU weight freezes demonstrated a clear decrease in training time per batch, although the sample size was quite small. OABU also demonstrated a decrease in inference time as well. These efficiencies also did not come with any decrease in accuracy as both ABU and OABU demonstrated similar testing accuracy.

It is clear that there are some benefits to using the adaptive blending units activation function, but one question remains. Outside of a research setting, is ABU a realistic choice for data scientists? For large models, OABU may work. Further research is needed to determine this however, and adding the extra complexity of ABU or OABU activation instead of ReLU may seem too risky for data scientists. However in the rare case of a team with a limited budget cannot find an optimal activation function after a few rounds of training, they may opt for an adaptable activation function such as OABU. Perhaps if there is a team with a very limited budget that can only afford a short period to train a model, the safest option may be to use OABU instead of using ReLU with the chance of running into the dying ReLU problem and having to retrain with a different activation function. However, before data science teams begin to choose this function we must first understand this activation function more.

As seen in the section above, tuning the hyperparameters for the thresholds of the ABU weight freezes yielded different results. The final values of a weight freeze threshold of less than 0.0005 for 40 cycles, and a hard freeze of .85 resulted in three hard freezes and two soft freezes

of the ABU weights. While this may seem like an irritating addition of yet more hyperparameters a data scientist would need to fine tune, this may not be the case. For example the Nesterov Momentum optimizer requires its own hyperparameter, but after years of its use a standard or default value of 0.9 has been widely accepted as being effective. The more people that experiment with OABU and its three new hyperparameters, the closer we can get to finding their best default values.

Our understanding of OABU could also benefit from experimentation with different optimizers, learning rates and other hyperparameters. Perhaps OABU works well with one optimizer, but has disastrous results with another. Maybe OABU works well with L_1 normalization but works better with L_2 normalization. The original ABU experiments did try different optimizers, and got slightly different results for each. There really is no way of knowing what relationships OABU has to different hyperparameters and architectures until they are all tried. Perhaps OABU should have its own optimizer with its own unique hyperparameters, or use the same optimizer and learning rate as the rest of the model. OABU could also benefit from experimentation with having each neuron have its own ABU blend as opposed to each layer having its own blend.

There is another improvement that could potentially be made for OABU. The researchers who made the original standard ABU used a very particular blend of activation functions that encompasses a wide range of function shapes. Perhaps there is an even better blend, that uses a specific combination of four or five activation functions from Kunc and Klema's compiled list. To find the perfect blend for OABU may not even be possible.

Perhaps another researcher may find further optimizations for adaptive blending units. Perhaps this paper is just one step in a long pathway to decreasing the runtime and increasing the accuracy of this adaptable activation function until one day it may appear as a more attractive choice for activation function than it currently is.

There may yet be an even better adaptable activation function that does not rely on a blend of existing activation functions, but discerns the optimal shape of the activation function entirely from the inherent properties of the neural network and its training data. The reasoning behind this belief is a suspicion based on the shapes of the functions in figures 6, 12, and 13. All of the ABU and OABU shapes look like some variation of a ReLU family function or a 'ReLU like function'. Is this a result of the optimal activation function shape being like ReLU, or is it a result of the specific blend of existing activation functions looking like and including ReLU? Perhaps using activation functions that are known and understood is merely a limitation of AAFs, or an AAF that does not use known functions may be too inefficient and computationally expensive. Only further research can truthfully answer this question.

Part 6: Appendix

Code for the CNN with OABU Activation:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms
import numpy as np
from collections import deque

ABU_ITERATIONS = 40

# Adaptive Blending Unit with 4 activations: ReLU, Tanh, ELU, Swish
# When one weight reaches 0.85, it becomes 1.0 and others become 0.0 (hard freeze)
class AdaptiveBlendingUnit(nn.Module):
    def __init__(self):
        super().__init__()
        self.frozen = False #frozen weights
        self.hard_frozen = False # When one activation dominates (>= 0.85)
        self.dominant_idx = None # if one activation function is dominant, it becomes
the only one

        # Four blending parameters for ReLU, Tanh, ELU, Swish
        self.weights = nn.Parameter(torch.ones(4) / 4) # Initialize equally

        # Track parameter history for freezing decision
        self.param_history = deque(maxlen=ABU_ITERATIONS)

    def forward(self, x):
        # Check if input is 2D (fully connected) or 4D (convolutional)
        is_2d = x.dim() == 2

        # If hard frozen, use only the dominant activation
        if self.hard_frozen and self.dominant_idx is not None:
            if self.dominant_idx == 0: # ReLU
                return F.relu(x)
            elif self.dominant_idx == 1: # Tanh
                return torch.tanh(x)
            elif self.dominant_idx == 2: # ELU
                return F.elu(x)
            elif self.dominant_idx == 3: # Swish
                return x * torch.sigmoid(x)
```

```

# Apply softmax to ensure weights sum to 1
if is_2d:
    w = F.softmax(self.weights, dim=0).view(1, 1, 4)
else:
    w = F.softmax(self.weights, dim=0).view(1, 1, 4, 1, 1)

# Compute all four activations
if is_2d:
    relu_out = F.relu(x).unsqueeze(2)
    tanh_out = torch.tanh(x).unsqueeze(2)
    elu_out = F.elu(x).unsqueeze(2)
    swish_out = (x * torch.sigmoid(x)).unsqueeze(2)
else:
    relu_out = F.relu(x).unsqueeze(2)
    tanh_out = torch.tanh(x).unsqueeze(2)
    elu_out = F.elu(x).unsqueeze(2)
    swish_out = (x * torch.sigmoid(x)).unsqueeze(2)

# Stack activations
activations = torch.cat([relu_out, tanh_out, elu_out, swish_out], dim=2)

# Weighted blend
output = (activations * w).sum(dim=2)

return output

def get_weight_values(self):
    # Return current weight values (after softmax)
    return F.softmax(self.weights, dim=-1).detach()

def check_and_freeze(self, threshold=0.0005):
    # Check if weight values have stabilized and freeze if so
    # Also check if any weight >= 0.85 for hard freeze
    if self.frozen:
        return True

    current_weights = self.get_weight_values()

    # Check for hard freeze condition (any weight >= 0.85)
    max_weight = torch.max(current_weights).item()
    if max_weight >= 0.85:
        self.hard_freeze(current_weights)
        return True

    self.param_history.append(current_weights.cpu().numpy())

    # Need at least ABU_ITERATIONS iterations to check stability
    if len(self.param_history) < ABU_ITERATIONS:
        return False

    # Calculate change over last ABU_ITERATIONS iterations
    weight_array = np.array(list(self.param_history))
    max_change = np.max(np.abs(weight_array[-1] - weight_array[0]))

    if max_change < threshold:

```

```

        # Check again if any weight is >= 0.85 before normal freeze
        if max_weight >= 0.85:
            self.hard_freeze(current_weights)
        else:
            self.freeze()
        return True

    return False

def hard_freeze(self, current_weights):
    #Hard freeze: Set dominant weight to 1.0, others to 0.0
    self.frozen = True
    self.hard_frozen = True
    self.dominant_idx = torch.argmax(current_weights).item()

    # Set weights: dominant = large value, others = very small
    # Use raw weights (before softmax) to create near one-hot after softmax
    with torch.no_grad():
        new_weights = torch.full_like(self.weights, -10.0) # Very negative
        new_weights[self.dominant_idx] = 10.0 # Very positive
        self.weights.copy_(new_weights)

    self.weights.requires_grad = False

    activation_names = ['ReLU', 'Tanh', 'ELU', 'Swish']
    weight_vals = self.get_weight_values().cpu().numpy()

    print(f"  HARD FROZEN ABU - {activation_names[self.dominant_idx]} reached >=
0.85")
    print(f"    Final weights: ReLU={weight_vals[0]:.4f},
Tanh={weight_vals[1]:.4f}, "
          f"ELU={weight_vals[2]:.4f}, Swish={weight_vals[3]:.4f}")

    def freeze(self):
        #Normal freeze: Stop training but keep current weight distribution
        self.frozen = True
        self.weights.requires_grad = False
        weight_vals = self.get_weight_values().cpu().numpy()

        print(f"  Frozen ABU with weights - ReLU: {weight_vals[0]:.4f}, "
              f"Tanh: {weight_vals[1]:.4f}, ELU: {weight_vals[2]:.4f}, Swish:
{weight_vals[3]:.4f}")

    def unfreeze(self):
        self.frozen = False
        self.hard_frozen = False
        self.dominant_idx = None
        self.weights.requires_grad = True

# CNN with ABU activations
class CIFAR10_ABU_Net(nn.Module):
    def __init__(self):
        super().__init__()

```



```

# Convolutional layers
self.conv1 = nn.Conv2d(3, 64, 3, padding=1)
self.bn1 = nn.BatchNorm2d(64)
self.abu1 = AdaptiveBlendingUnit()

self.conv2 = nn.Conv2d(64, 128, 3, padding=1)
self.bn2 = nn.BatchNorm2d(128)
self.abu2 = AdaptiveBlendingUnit()

self.conv3 = nn.Conv2d(128, 256, 3, padding=1)
self.bn3 = nn.BatchNorm2d(256)
self.abu3 = AdaptiveBlendingUnit()

self.conv4 = nn.Conv2d(256, 512, 3, padding=1)
self.bn4 = nn.BatchNorm2d(512)
self.abu4 = AdaptiveBlendingUnit()

# Fully connected layers
self.fc1 = nn.Linear(512 * 2 * 2, 512)
self.abu5 = AdaptiveBlendingUnit()
self.dropout = nn.Dropout(0.5)
self.fc2 = nn.Linear(512, 10)

self.pool = nn.MaxPool2d(2, 2)

def forward(self, x):
    # Conv block 1
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.abu1(x)
    x = self.pool(x)

    # Conv block 2
    x = self.conv2(x)
    x = self.bn2(x)
    x = self.abu2(x)
    x = self.pool(x)

    # Conv block 3
    x = self.conv3(x)
    x = self.bn3(x)
    x = self.abu3(x)
    x = self.pool(x)

    # Conv block 4
    x = self.conv4(x)
    x = self.bn4(x)
    x = self.abu4(x)
    x = self.pool(x)

    # Fully connected
    x = x.view(x.size(0), -1)
    x = self.fc1(x)
    x = self.abu5(x)
    x = self.dropout(x)

```

```

        x = self.fc2(x)

        return x

def get_all_abus(self):
    # Return all ABU modules
    return [self.abu1, self.abu2, self.abu3, self.abu4, self.abu5]

def check_and_freeze_abus(self, threshold=0.0001):
    # Check all ABUs and freeze those that have stabilized
    frozen_count = 0
    for i, abu in enumerate(self.get_all_abus()):
        was_frozen = abu.frozen
        if abu.check_and_freeze(threshold):
            if abu.frozen and not was_frozen: # Just frozen
                print(f"Layer {i + 1} ABU frozen")
                frozen_count += 1
    return frozen_count

def train_epoch(model, trainloader, criterion, optimizer, device, epoch,
abu_freeze_threshold=0.0005):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for i, (inputs, labels) in enumerate(trainloader):
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

        # Check ABU stability every 2 batches
        if i % 2 == 1:
            frozen_count = model.check_and_freeze_abus(threshold=abu_freeze_threshold)

        if i % 100 == 99:
            frozen_count = sum(1 for abu in model.get_all_abus() if abu.frozen)
            hard_frozen_count = sum(1 for abu in model.get_all_abus() if
abu.hard_frozen)
            print(f'  Batch {i + 1}, Loss: {running_loss / 100:.3f}, '
                  f'Acc: {100. * correct / total:.2f}%, Frozen ABUs: {frozen_count}/5
,
                  f'(Hard: {hard_frozen_count})')
            running_loss = 0.0

```

```

    return correct / total

def validate(model, testloader, criterion, device):
    model.eval()
    correct = 0
    total = 0
    val_loss = 0.0

    with torch.no_grad():
        for inputs, labels in testloader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)

            val_loss += loss.item()
            _, predicted = outputs.max(1)
            total += labels.size(0)
            correct += predicted.eq(labels).sum().item()

    return correct / total, val_loss / len(testloader)

def main():
    # Hyperparameters
    batch_size = 128
    num_epochs = 50
    learning_rate = 0.001
    abu_freeze_threshold = 0.0005 # Threshold for freezing ABUs

    # Device
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f'Using device: {device}')

    # Data transforms
    transform_train = transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
    ])

    transform_test = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
    ])

    # Load CIFAR-10
    print('Loading CIFAR-10 dataset...')
    trainset = datasets.CIFAR10(root='./data', train=True, download=True,
                                transform=transform_train)
    trainloader = DataLoader(trainset, batch_size=batch_size, shuffle=True,
                             num_workers=2)

    testset = datasets.CIFAR10(root='./data', train=False, download=True,

```

```

        transform=transform_test)
testloader = DataLoader(testset, batch_size=batch_size, shuffle=False,
                        num_workers=2)

# Model, loss, optimizer
print('Initializing model...')
model = CIFAR10_ABU_Net().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=num_epochs)

# Training loop

# summary(model, input_size=(3, 32, 32))
print('\nStarting training...')
print('=' * 70)

best_acc = 0.0

for epoch in range(num_epochs):
    print(f'\nEpoch {epoch + 1}/{num_epochs}')
    print('-' * 70)

    # Train
    train_acc = train_epoch(model, trainloader, criterion, optimizer, device,
epoch, abu_freeze_threshold)

    # Validate
    val_acc, val_loss = validate(model, testloader, criterion, device)

    frozen_count = sum(1 for abu in model.get_all_abus() if abu.frozen)
    hard_frozen_count = sum(1 for abu in model.get_all_abus() if abu.hard_frozen)
    print(f'Train Acc: {100. * train_acc:.2f}%, Val Acc: {100. * val_acc:.2f}%, '
          f'Val Loss: {val_loss:.3f}, Frozen ABUs: {frozen_count}/5 (Hard:
{hard_frozen_count})')

    # Display current alpha values every 5 epochs
    if (epoch + 1) % 5 == 0:
        print('\nCurrent ABU weights (ReLU, Tanh, ELU, Swish):')
        for i, abu in enumerate(model.get_all_abus()):
            weight_vals = abu.get_weight_values()
            if abu.hard_frozen:
                status = "HARD FROZEN"
            elif abu.frozen:
                status = "FROZEN"
            else:
                status = "active"
            print(f' Layer {i + 1}: ReLU={weight_vals[0].item():.4f}, '
                  f'Tanh={weight_vals[1].item():.4f}, '
                  f'ELU={weight_vals[2].item():.4f}, '
                  f'Swish={weight_vals[3].item():.4f} [{status}]')

    # Update learning rate
    scheduler.step()

```

```

# Save best model
if val_acc > best_acc:
    best_acc = val_acc
    torch.save({
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'best_acc': best_acc,
    }, 'cifar10_abu_best.pth')
    print(f' → Saved new best model (acc: {100. * best_acc:.2f}%)')

print('\n' + '=' * 70)
print(f'Training complete! Best validation accuracy: {100. * best_acc:.2f}%)')

# Final ABU statistics
print('\nFinal ABU Status:')
print('-' * 70)
activation_names = ['ReLU', 'Tanh', 'ELU', 'Swish']
for i, abu in enumerate(model.get_all_abus()):
    weight_vals = abu.get_weight_values()
    if abu.hard_frozen:
        status = "HARD FROZEN"
        dominant = activation_names[abu.dominant_idx]
        print(f'Layer {i + 1}: {dominant} = 1.0 (others = 0.0) [{status}]')
    elif abu.frozen:
        status = "FROZEN"
        print(f'Layer {i + 1}: ReLU={weight_vals[0].item():.4f}, '
              f'Tanh={weight_vals[1].item():.4f}, '
              f'ELU={weight_vals[2].item():.4f}, '
              f'Swish={weight_vals[3].item():.4f} [{status}]')
    else:
        status = "ACTIVE"
        print(f'Layer {i + 1}: ReLU={weight_vals[0].item():.4f}, '
              f'Tanh={weight_vals[1].item():.4f}, '
              f'ELU={weight_vals[2].item():.4f}, '
              f'Swish={weight_vals[3].item():.4f} [{status}]')

if __name__ == '__main__':
    main()

```

Code for the Standard ABU CNN:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# Adaptive Blending Unit Class
class AdaptiveBlendingUnit(nn.Module):
    def __init__(self):
        super().__init__()
        # Five blending parameters for ReLU, Tanh, ELU, Swish, Identity
        self.weights = nn.Parameter(torch.ones(5) / 5) # Initialize equally

    def forward(self, x):
        # Check if input is 2D (fully connected) or 4D (convolutional)
        is_2d = x.dim() == 2

        # Apply softmax to ensure weights sum to 1
        if is_2d:
            w = F.softmax(self.weights, dim=0).view(1, 1, 5)
        else:
            w = F.softmax(self.weights, dim=0).view(1, 1, 5, 1, 1)

        # Compute all five activations
        if is_2d:
            relu_out = F.relu(x).unsqueeze(2)
            tanh_out = torch.tanh(x).unsqueeze(2)
            elu_out = F.elu(x).unsqueeze(2)
            swish_out = (x * torch.sigmoid(x)).unsqueeze(2)
            identity_out = x.unsqueeze(2)
        else:
            relu_out = F.relu(x).unsqueeze(2)
            tanh_out = torch.tanh(x).unsqueeze(2)
            elu_out = F.elu(x).unsqueeze(2)
            swish_out = (x * torch.sigmoid(x)).unsqueeze(2)
            identity_out = x.unsqueeze(2)

        # Stack activations
        activations = torch.cat([relu_out, tanh_out, elu_out, swish_out,
            identity_out], dim=2)

        # Weighted blend
        output = (activations * w).sum(dim=2)

        return output
```

```

def get_weight_values(self):
    # Return current weight values (after softmax)
    return F.softmax(self.weights, dim=-1).detach()

# CNN with ABU activations
class CIFAR10_ABU_Net(nn.Module):
    def __init__(self):
        super().__init__()
        # Convolutional layers
        self.conv1 = nn.Conv2d(3, 64, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(64)
        self.abu1 = AdaptiveBlendingUnit()

        self.conv2 = nn.Conv2d(64, 128, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(128)
        self.abu2 = AdaptiveBlendingUnit()

        self.conv3 = nn.Conv2d(128, 256, 3, padding=1)
        self.bn3 = nn.BatchNorm2d(256)
        self.abu3 = AdaptiveBlendingUnit()

        self.conv4 = nn.Conv2d(256, 512, 3, padding=1)
        self.bn4 = nn.BatchNorm2d(512)
        self.abu4 = AdaptiveBlendingUnit()

        # Fully connected layers
        self.fc1 = nn.Linear(512 * 2 * 2, 512)
        self.abu5 = AdaptiveBlendingUnit()
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(512, 10)

        self.pool = nn.MaxPool2d(2, 2)

    def forward(self, x):
        # Conv block 1
        x = self.conv1(x)
        x = self.bn1(x)
        x = self.abu1(x)
        x = self.pool(x)

        # Conv block 2
        x = self.conv2(x)
        x = self.bn2(x)
        x = self.abu2(x)
        x = self.pool(x)

        # Conv block 3

```

```

        x = self.conv3(x)
        x = self.bn3(x)
        x = self.abu3(x)
        x = self.pool(x)

        # Conv block 4
        x = self.conv4(x)
        x = self.bn4(x)
        x = self.abu4(x)
        x = self.pool(x)

        # Fully connected
        x = x.view(x.size(0), -1)
        x = self.fc1(x)
        x = self.abu5(x)
        x = self.dropout(x)
        x = self.fc2(x)

    return x

def get_all_abus(self):
    # Return all ABU modules
    return [self.abu1, self.abu2, self.abu3, self.abu4, self.abu5]

def train_epoch(model, trainloader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for i, (inputs, labels) in enumerate(trainloader):
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

    if i % 100 == 99:
        print(f' Batch {i + 1}, Loss: {running_loss / 100:.3f}, '
              f'Acc: {100. * correct / total:.2f}%')
        running_loss = 0.0

```



```

    return correct / total

def validate(model, testloader, criterion, device):
    model.eval()
    correct = 0
    total = 0
    val_loss = 0.0

    with torch.no_grad():
        for inputs, labels in testloader:
            inputs, labels = inputs.to(device), labels.to(device)
            outputs = model(inputs)
            loss = criterion(outputs, labels)

            val_loss += loss.item()
            _, predicted = outputs.max(1)
            total += labels.size(0)
            correct += predicted.eq(labels).sum().item()

    return correct / total, val_loss / len(testloader)

def main():
    # Hyperparameters
    batch_size = 128
    num_epochs = 50
    learning_rate = 0.001

    # Device
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f'Using device: {device}')

    # Data transforms
    transform_train = transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994,
0.2010)),
    ])

    transform_test = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994,
0.2010)),
    ])

```

```

# Load CIFAR-10
print('Loading CIFAR-10 dataset...')
trainset = datasets.CIFAR10(root='./data', train=True, download=True,
                             transform=transform_train)
trainloader = DataLoader(trainset, batch_size=batch_size, shuffle=True,
num_workers=2)

testset = datasets.CIFAR10(root='./data', train=False, download=True,
                             transform=transform_test)
testloader = DataLoader(testset, batch_size=batch_size, shuffle=False,
num_workers=2)

# Model, loss, optimizer
print('Initializing model...')
model = CIFAR10_ABU_Net().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=num_epochs)

# Training loop
print('\nStarting training...')
print('=' * 70)
best_acc = 0.0

for epoch in range(num_epochs):
    print(f'\nEpoch {epoch + 1}/{num_epochs}')
    print('-' * 70)

    # Train
    train_acc = train_epoch(model, trainloader, criterion, optimizer,
device)

    # Validate
    val_acc, val_loss = validate(model, testloader, criterion, device)

    print(f'Train Acc: {100. * train_acc:.2f}%, Val Acc: {100. *
val_acc:.2f}%, '
          f'Val Loss: {val_loss:.3f}')

# Display current ABU weights every 5 epochs
if (epoch + 1) % 5 == 0:
    print('\nCurrent ABU weights (ReLU, Tanh, ELU, Swish, Identity):')
    for i, abu in enumerate(model.get_all_abus()):
        weight_vals = abu.get_weight_values()
        print(f' Layer {i + 1}: ReLU={weight_vals[0].item():.4f}, '
              f'Tanh={weight_vals[1].item():.4f}, '
              f'ELU={weight_vals[2].item():.4f}, '
              f'Swish={weight_vals[3].item():.4f}, '

```

```

        f'Identity={weight_vals[4].item():.4f}'))

# Update learning rate
scheduler.step()

# Save best model
if val_acc > best_acc:
    best_acc = val_acc
    torch.save({
        'epoch': epoch,
        'model_state_dict': model.state_dict(),
        'optimizer_state_dict': optimizer.state_dict(),
        'best_acc': best_acc,
    }, 'cifar10_abu_best.pth')
    print(f' → Saved new best model (acc: {100. * best_acc:.2f}%)')

print('\n' + '=' * 70)
print(f'Training complete! Best validation accuracy: {100. *
best_acc:.2f}%)')

# Final ABU statistics
print('\nFinal ABU Weights:')
print('-' * 70)
for i, abu in enumerate(model.get_all_abus()):
    weight_vals = abu.get_weight_values()
    print(f'Layer {i + 1}: ReLU={weight_vals[0].item():.4f}, '
          f'Tanh={weight_vals[1].item():.4f}, '
          f'ELU={weight_vals[2].item():.4f}, '
          f'Swish={weight_vals[3].item():.4f}, '
          f'Identity={weight_vals[4].item():.4f}')

if __name__ == '__main__':
    main()

```

Code for the ReLU Control Group:

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim
from torch.utils.data import DataLoader
from torchvision import datasets, transforms

# CNN with ReLU activations
class CIFAR10_Net(nn.Module):
    def __init__(self):
        super().__init__()
        # Convolutional layers
        self.conv1 = nn.Conv2d(3, 64, 3, padding=1)
        self.bn1 = nn.BatchNorm2d(64)

        self.conv2 = nn.Conv2d(64, 128, 3, padding=1)
        self.bn2 = nn.BatchNorm2d(128)

        self.conv3 = nn.Conv2d(128, 256, 3, padding=1)
        self.bn3 = nn.BatchNorm2d(256)

        self.conv4 = nn.Conv2d(256, 512, 3, padding=1)
        self.bn4 = nn.BatchNorm2d(512)

        # Fully connected layers
        self.fc1 = nn.Linear(512 * 2 * 2, 512)
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(512, 10)

        self.pool = nn.MaxPool2d(2, 2)

    def forward(self, x):
        # Conv block 1
        x = self.conv1(x)
        x = self.bn1(x)
        x = F.relu(x)
        x = self.pool(x)

        # Conv block 2
        x = self.conv2(x)
        x = self.bn2(x)
        x = F.relu(x)
        x = self.pool(x)

        # Conv block 3
        x = self.conv3(x)
```

```

        x = self.bn3(x)
        x = F.relu(x)
        x = self.pool(x)

        # Conv block 4
        x = self.conv4(x)
        x = self.bn4(x)
        x = F.relu(x)
        x = self.pool(x)

        # Fully connected
        x = x.view(x.size(0), -1)
        x = self.fcl(x)
        x = F.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)

    return x

def train_epoch(model, trainloader, criterion, optimizer, device):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for i, (inputs, labels) in enumerate(trainloader):
        inputs, labels = inputs.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

    if i % 100 == 99:
        print(f'  Batch {i + 1}, Loss: {running_loss / 100:.3f}, '
              f'Acc: {100. * correct / total:.2f}%')
        running_loss = 0.0

    return correct / total

def validate(model, testloader, criterion, device):

```

```

model.eval()
correct = 0
total = 0
val_loss = 0.0

with torch.no_grad():
    for inputs, labels in testloader:
        inputs, labels = inputs.to(device), labels.to(device)
        outputs = model(inputs)
        loss = criterion(outputs, labels)

        val_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()

    return correct / total, val_loss / len(testloader)

def main():
    # Hyperparameters
    batch_size = 128
    num_epochs = 50
    learning_rate = 0.001

    # Device
    device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
    print(f'Using device: {device}')

    # Data transforms
    transform_train = transforms.Compose([
        transforms.RandomCrop(32, padding=4),
        transforms.RandomHorizontalFlip(),
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994,
0.2010)),
    ])

    transform_test = transforms.Compose([
        transforms.ToTensor(),
        transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994,
0.2010)),
    ])

    # Load CIFAR-10
    print('Loading CIFAR-10 dataset...')
    trainset = datasets.CIFAR10(root='./data', train=True, download=True,
                                transform=transform_train)

```

```

trainloader = DataLoader(trainset, batch_size=batch_size, shuffle=True,
num_workers=2)

testset = datasets.CIFAR10(root='./data', train=False, download=True,
                           transform=transform_test)
testloader = DataLoader(testset, batch_size=batch_size, shuffle=False,
num_workers=2)

# Model, loss, optimizer
print('Initializing model...')
model = CIFAR10_Net().to(device)
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=learning_rate)
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer,
T_max=num_epochs)

# Training loop
print('\nStarting training...')
print('=' * 70)
best_acc = 0.0

for epoch in range(num_epochs):
    print(f'\nEpoch {epoch + 1}/{num_epochs}')
    print('-' * 70)

    # Train
    train_acc = train_epoch(model, trainloader, criterion, optimizer,
device)

    # Validate
    val_acc, val_loss = validate(model, testloader, criterion, device)

    print(f'Train Acc: {100. * train_acc:.2f}%, Val Acc: {100. *
val_acc:.2f}%, '
          f'Val Loss: {val_loss:.3f}%)

    # Update learning rate
    scheduler.step()

    # Save best model
    if val_acc > best_acc:
        best_acc = val_acc
        torch.save({
            'epoch': epoch,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'best_acc': best_acc,
        }, 'cifar10_best.pth')
    print(f' → Saved new best model (acc: {100. * best_acc:.2f}%)')

```

```
    print('\n' + '=' * 70)
    print(f'Training complete! Best validation accuracy: {100. *
best_acc:.2f}%')

if __name__ == '__main__':
    main()
```


Bibliography:

- V. Kunc and J. Kléma, "Three decades of activations: A comprehensive survey of 400 activation functions for neural networks," *arXiv*, preprint arXiv:2402.09092, Feb. 2024, doi: 10.48550/arXiv.2402.09092.
- Chen. B, "What do you mean by saturation in the context of neural network training? Discuss the problems associated with saturation," AIML.com, Apr. 2025. [Online]. Available: <https://aiml.com/what-do-you-mean-by-saturation-in-the-context-of-neural-network-training-discuss-the-problems-associated-with-saturation/>
- Ritika Dokania, "What is the dead relu problem and, why is it an issue in neural network training?" AIML.com. (2024, July 22). <https://aiml.com/what-is-dying-relu-or-dead-relu-and-why-is-this-a-problem-in-neural-network-training/>
- S. C. Douglas and J. Yu, "Why RELU Units Sometimes Die: Analysis of Single-Unit Error Backpropagation in Neural Networks," 2018 52nd Asilomar Conference on Signals, Systems, and Computers, Pacific Grove, CA, USA, 2018, pp. 864-868, doi: 10.1109/ACSSC.2018.8645556. keywords: {Training;Convergence;Analytical models;Computer architecture;Neural networks;Machine learning;Probability;adaptive algorithms;adaptation models;algorithm design and analysis;backpropagation algorithms},
- M. A. Mercioni and S. Holban, "P-Swish: Activation Function with Learnable Parameters Based on Swish Activation Function in Deep Learning," 2020 International Symposium on Electronics and Telecommunications (ISETC), Timisoara, Romania, 2020, pp. 1-4, doi: 10.1109/ISETC50328.2020.9301059. keywords: {Task analysis;Deep learning;Proposals;Natural language processing;Neurons;Mathematical model;Logistics;activation function;Cifar-10;Cifar-100;Deep Learning;learnable;Machine Learning;Natural Language Processing;ReLU;Swish},
- L. R. Sütfield and F. Brieger, "Adaptive blending units: Trainable activation functions for deep neural networks," arXiv:1806.10064, Jun. 2018. [Online]. Available: <https://arxiv.org/abs/1806.10064>
- M. D. Zeiler and R. Fergus, "Visualizing and understanding convolutional networks," arXiv:1311.2901, Nov. 2013. [Online]. Available: <https://arxiv.org/abs/1311.2901>